

Оглавление

1	Манипулирование реляционными данными	2
1.1	Методы манипулирования реляционными данными	2
1.2	Операции реляционной алгебры	2
1.2.1	Операторы и приоритеты	2
1.2.2	Объединение (UNION)	3
1.2.3	Пересечение (INTERSECT).....	3
1.2.4	Разность (MINUS)	3
1.2.5	Декартово произведение (TIMES).....	4
1.2.6	Выборка (WHERE).....	5
1.2.7	Проекция (PROJECT)	5
1.2.8	Соединение (JOIN).....	6
1.2.9	Деление (DIVIDE BY)	7
1.3	Дополнения.....	7
1.3.1	Дополнительные операции	7
1.3.2	Расширение реляционной алгебры. Алгебра A.....	8
1.4	Свойства операций	9
1.5	Реляционное исчисление.....	9
1.5.1	Исчисление кортежей	9
1.5.2	Исчисление доменов	10
2	Язык SQL – манипулирование данными	11
2.1	Структура запроса.....	11
2.2	Запросы с условием	13
2.3	Запросы с агрегированием	14
2.4	Запросы с соединением. Виды соединений	15

1 МАНИПУЛИРОВАНИЕ РЕЛЯЦИОННЫМИ ДАННЫМИ

1.1 Методы манипулирования реляционными данными

Реляционная алгебра

Выражения строятся на основе алгебраических операций (высокого уровня) имеют процедурную интерпретацию (процедурный язык)

Другими словами, запрос, представленный на языке реляционной алгебры, может быть вычислен на основе выполнения элементарных алгебраических операций с учетом их приоритетности и возможного наличия скобок.

Реляционное исчисление

Подразделяется на исчисление кортежей и исчисление доменов. Для каждой формулы реляционного исчисления существует однозначная вычислительная интерпретация. Формула не определяет способ получения результата, а ставит условия, которым должны удовлетворять кортежи результирующего отношения.

Поэтому языки реляционного исчисления являются в большей степени непроцедурными, или декларативными.

Язык манипулирования данными в РБД является реляционно-полным, если любой корректный запрос, формулируемой с помощью одного выражения реляционной алгебры (или одной формулы реляционного исчисления), может быть сформулирован с помощью одного оператора этого языка.

1.2 Операции реляционной алгебры

1.2.1 Операторы и приоритеты

Операторы реляционной алгебры выполняются над отношениями.

Свойство реляционных операций - **замкнутость** (относительно понятия отношения). Это означает, что выражения реляционной алгебры определяются над отношениями реляционных БД и результатом вычисления также являются отношения.

Реляционная алгебра Кодда включает в себя 8 операций из двух групп:

1. Теоретико-множественные операции.
 - a. объединение; (UNION)
 - b. пересечение; (INTERSECT)
 - c. разность; (MINUS)
 - d. декартово произведение. (TIMES)
2. Специальные реляционные операции.
 - a. выборку (ограничение); (WHERE)
 - b. проекцию; (PROJECT)
 - c. соединение; (JOIN)
 - d. деление. (DIVIDE BY)

Операции *объединения, пересечения и разности* требуют от операндов **совместимости по типу**. Два отношения совместимы по типу, если:

1. Их заголовки имеют одинаковое множество атрибутов.
2. Одинаковые атрибуты имеют одинаковые длины.

Поскольку результатом любой реляционной операции (*кроме операции присваивания, которая не вырабатывает значения*) является некое отношение, можно образовывать реляционные выражения, в которых вместо отношения-операнда некоторой реляционной операции находится вложенное реляционное выражение. В построении реляционного выражения могут участвовать все реляционные операции, кроме операции

присваивания. Вычислительная интерпретация реляционного выражения диктуется установленными приоритетами операций:

$\text{RENAME} \geq \text{WHERE} = \text{PROJECT} \geq \text{TIMES} = \text{JOIN} = \text{INTERSECT}$
 $= \text{DIVIDE BY} \geq \text{UNION} = \text{MINUS}$

Операция	Приоритет
RENAME	4
WHERE	3
PROJECT	3
TIMES	2
JOIN	2
INTERSECT	2
DIVIDE BY	2
UNION	1
MINUS	1

1.2.2 Объединение (UNION)

Результатом соединения отношения A и B является отношение C с тем же заголовком, что и у операндов, и телом из кортежей, принадлежащим A или B или обоим.
 $A \text{ UNION } B \quad R=R1 \cup R2$

	A_1	A_2		A_1	A_2		A_1	A_2
R_1	1	2		5	6		1	2
	3	4	R_2	1	2		3	4
							5	6

=>

	A_1	A_2
R	1	2
	3	4
	5	6

Обратите внимание, что повторяющиеся кортежи удаляются по определению отношения (в SQL это не так).

1.2.3 Пересечение (INTERSECT)

Результатом соединения отношения A и B является отношение C с тем же заголовком, что и у операндов, и телом из кортежей, принадлежащим A и B одновременно.
 $A \text{ INTERSECT } B \quad R=R1 \cap R2$

	A_1	A_2		A_1	A_2		A_1	A_2
R_1	1	2		5	6		3	4
	3	4	R_2	1	2			

=>

	A_1	A_2
R	3	4

1.2.4 Разность (MINUS)

Результатом соединения отношения A и B является отношение C с тем же заголовком, что и у операндов, и телом из кортежей, принадлежащим A и не принадлежащим B
 $A \text{ MINUS } B \quad R=R1 - R2$

Отношение A

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21

Отношение B

ID_NUM	NAME	CITY	AGE
1809	Иванов	Москва	45
1896	Галкин	Иваново	40

A - B

ID_NUM	NAME	CITY	AGE
1996	Петров	Нижний Новгород	39
1777	Сидоров	Рязань	21

1.2.5 Декартово произведение (TIMES)

В теории множеств декартово произведение может быть получено для любых двух множеств, и элементами результирующего множества являются пары, составленные из элементов первого и второго множеств.

Декартовым произведением **множеств** $A\{a\}$ и $B\{b\}$ является такое множество пар $C\{<c1, c2>\}$, что для каждого элемента $<c1, c2>$ множества C существуют такой элемент a множества A , что $c1=a$, и такой элемент b множества B , что $c2=b$.

Поскольку отношения являются множествами, для любых двух отношений возможно получение прямого произведения. Но результат не будет отношением! Элементами результата будут не кортежи, а пары кортежей.

Поэтому в реляционной алгебре используется специализированная форма операции взятия декартова произведения – **расширенное декартово произведение отношений**. Оно применимо к отношениям, заголовки которых *не содержат одноименных атрибутов одного домена*.

Два отношения *совместимы по взятию расширенного декартова произведения* в том и только в том случае, если пересечение множеств имен атрибутов, взятых из их схем отношений, пусто.

Результат **декартова произведения отношения** A с заголовком $A(A_1, \dots, A_m)$ и отношения $B(B_1, \dots, B_n)$ является отношение $C=A \text{ TIMES } B$, заголовок которого является конкатенацией заголовков операндов $C(A_1, \dots, A_m, B_1, \dots, B_n)$, а тело состоит из кортежей вида $(a_1, \dots, a_m, b_1, \dots, b_n)$, где $(a_1, \dots, a_m) \in A$, $(b_1, \dots, b_n) \in B$.

Кардинальное число (число кортежей) результирующего отношения равно произведению кардинальных чисел исходных отношений, а *степень* равняется сумме степеней.

$$t=R \times S = R \otimes S$$

	A_1	A_2		A_1	A_3
R	1	2	S	5	6
	3	4		7	8

	$R. A_2$	A_2	$S. A_1$	A_3
t	1	2	5	6
	1	2	7	8
	3	4	5	6
	3	4	7	8

Для достижения совместимости используется специально введенная операция *переименования*

В общем виде декартово произведение некоммукативно. **Коммутативным** является декартово произведение, определенное на *классах эквивалентности по заданному отношению*.

Реляционные отношения A и B **эквивалентны**, если:

1. Их заголовки совпадают.
2. Для каждого кортежа, принадлежащего A , найдется равный ему кортеж отношения B , и наоборот.

На практике явное использование операции декартового произведения требуется только для очень сложных запросов. Эта операция включена в реляционную алгебру по концептуальным соображениям: декартово произведение требуется как промежуточный шаг при определении операции соединения, которая используется довольно часто.

1.2.6 Выборка (WHERE)

Операция выборки также иногда называется операцией ограничения или операцией селекции. Это унарная операция, результатом применения которой к отношению A является отношение с тем же заголовком, что и A , тело которого состоит из кортежей, в которые входят атрибуты A , удовлетворяющие условию p .

$A \text{ WHERE } p$.

$t = \sigma_p(A)$

	A_1	A_2
R	1	2
	9	8
	3	3

	A_1	A_2
$t = \sigma_{A_1 \leq A_2}(R)$	1	2
	3	3

Выражение вычислимо **в трехзначной логике**: true, false, unknown.

1.2.7 Проекция (PROJECT)

Результатом проекции отношения A по заданному набору атрибутов $(a_1, \dots, a_m)$ ($A(a_1, \dots, a_n)$), $\forall a \in (a_1, \dots, a_m)$ является отношение с заголовком $(a_1, \dots, a_m)$ и телом из соответствующих фрагментов кортежей отношения A .

$\text{PROJECT}(A(a_1, \dots, a_m))$

$t = \pi_{A_1, A_4}(A)$

	A_1	A_2	A_3	A_4
R	1	2	3	4
	7	8	9	10
	3	4	5	6
	3	4	7	6

	A_1	A_4
$t = \Pi_{A_1, A_4}(R)$	1	4
	7	10
	3	6

1.2.8 Соединение (JOIN)

Операция формирует отношение из объединения кортежей своих операндов. Существует несколько видов соединения, основанных на различных вариантах соединения по условию.

Соединение по условию

Результатом соединения отношений A и B по условию, заданному логическим выражением p , является отношение R, заголовок которого совпадает с заголовком декартова произведения отношений A и B, а тело содержит множество кортежей t , таких что каждый из них принадлежит декартову произведению A и B и условие p для них истинно.

$$t = \sigma_p(R \times S) = R \bowtie_p S$$

Эквисоединение

То же самое, что и соединение по условию, только условие p задает равенство операндов. A EQUIJOIN B ($A.a=B.b$) или аналогично соединению по условию $t = \sigma_p(R \times S) = R \bowtie_p S$

Естественное соединение

Применяется к двум отношениям, имеющим общий атрибут (простой или составной). A NATURAL JOIN B или $t = R \bowtie S$.

Определение этой операции следует из способа построения естественного соединения.

	A_1	A_2	A_3
R	1	2	3
	4	6	7

	A_1	A_2	A_4	A_5
S	1	2	7	8
	8	9	10	11
	4	6	9	16

Построение естественного соединения:

1. построить декартово произведение $R \times S$

	$R.A_1$	$R.A_2$	A_3	$S.A_1$	$S.A_2$	A_4	A_5
t_1	1	2	3	1	2	7	8
	1	2	3	8	9	10	11
	1	2	3	4	6	9	16
	4	6	7	1	2	7	8
	4	6	7	8	9	10	11
	4	6	7	4	6	9	16

2. выбрать из этого произведения кортежи по условию $R.A_{i1}=S.A_{i1} \dots R.A_{ik}=S.A_{ik}$, где $A_{i1} \dots A_{ik}$ - общие атрибуты в схемах отношений R и S

	$R.A_1$	$R.A_2$	A_3	$S.A_1$	$S.A_2$	A_4	A_5
t_2	1	2	3	1	2	7	8
	4	6	7	4	6	9	16

3. удалить из полученного отношения $S.A_1...S.A_k$, потому что они будут дублирующими.

	$R.A_1$	$R.A_2$	A_3	A_4	A_5
t_3	1	2	3	7	8
	4	6	7	9	16

В языке SQL есть левое и правое соединение.

1.2.9 Деление (DIVIDE BY)

В операции участвуют два отношения $A(a_1, ..., a_n, b_1, ..., b_m)$ и $B(b_1, ..., b_m)$, где $b_i \in H_A$ и $b_i \in H_B$ – определены на одном домене.

Результатом деления является отношение $C(a_1, ..., a_n)$, тело которого состоит из множества кортежей $(a_{1k}, a_{2k}, ..., a_{nk})$, таких, что для **всех** кортежей $(b_{1k}, b_{2k}, ..., b_{mk})$ из B в отношении A найдется кортеж $(a_{1k}, a_{2k}, ..., a_{nk}, b_{1k}, b_{2k}, ..., b_{mk})$.

$M \text{ DIVIDE BY } D$

M/D

Отношение A

S#	P#
s1	P1
s1	P2
s1	P3
s1	P4
s2	P1
s2	P3
s3	P2
s3	P3

Отношение B

P#
P1

Отношение B1

P#
P2
P3

Отношение B2

P#
P1
P2
P3

A/B

S#
s1
s2

A/B1

S#
s1
s3

A/B2

S#
s1

По сути, представляет собой алгебраический аналог квантора всеобщности (полезна тогда, когда запрос содержит слово «все»).

1.3 Дополнения

1.3.1 Дополнительные операции

Заголовок отношения представляет собой множество пар <имя-атрибута, имя-домена>. Если посмотреть на общий обзор реляционных операций, то видно, что домены атрибутов результирующего отношения однозначно определяются доменами отношений-операндов. Однако с именами атрибутов результата не всегда все так просто.

Например, представим себе, что у отношений-операндов операции декартова произведения имеются одноименные атрибуты с одинаковыми доменами. *Каким был бы заголовок результирующего отношения?*

Поскольку это множество, в нем не должны содержаться одинаковые элементы. Но и потерять атрибут в результате недопустимо. А это значит, что в таком случае вообще невозможно корректно выполнить операцию декартова произведения.

Аналогичные проблемы могут возникать и в случаях других двуместных операций.

Для разрешения проблем в число операций реляционной алгебры вводится **операция переименования**. Ее следует применять в том случае, когда возникает конфликт именования атрибутов в отношениях-операндах одной реляционной операции. Тогда к одному из операндов сначала применяется операция переименования, а затем основная операция выполняется уже без всяких проблем. Результатом этой операции является отношение, совпадающее во всем с отношением-операндом, кроме того, что имя указанного атрибута изменено на заданное имя.

Операция переименования атрибутов, дающая возможность корректно сформировать заголовок (схему) результирующего отношения (A RENAME X AS Y),

Также на практике с операциями алгебры используется ряд дополнительных операций (помимо переименования), основанных на понятии реляционной переменной:

Операция присваивания, позволяющая сохранить в базе данных результаты вычисления алгебраических выражений (A:= B),

Операция расширения, позволяющая создавать новое отношение, дополненное атрибутом, значения которого получены посредством некоторых скалярных вычислений (EXTEND <имя отношения> AS <скалярное выражение>),

Операция подведения итогов, дающая возможность разбивать множество кортежей отношения на группы в соответствии с содержимым одного или нескольких атрибутов, и внутри каждой группы применять определенный оператор агрегирования (аналог операции GROUP BY в языке SQL).

Операторы агрегирования предназначены для подведения итогов в определенном столбце таблицы – отношения, например, для нахождения суммарных, средних, минимальных и максимальных значений (SUM, AVG, MIN, MAX, COUNT).

SUM(QTY) AS TOTAL_QTY

1.3.2 Расширение реляционной алгебры. Алгебра А

Переосмысливая положения классической реляционной алгебры Кодда, Дейт и Дарвен пришли к, как они считают, более логичной (в математическом смысле) формулировке реляционной алгебры, названной ими “Алгеброй А”.

Название А представляет собой двойной рекурсивный акроним от ALGEBRA, что дополнительно раскрывается авторами как A Logical Genesis Explains Basic Relational Algebra. Как видно из развернутого названия, алгебра А построена таким образом, чтобы подчеркнуть, возможно, более отчетливо, чем это получалось в предыдущих алгебрах, насколько тесно реляционная алгебра связана с дисциплиной логики предикатов.

Базисом предложенной Кристофером Дейтом и Хьюго Дарвеном Алгебры А являются операции **реляционного отрицания** (дополнения), **реляционной конъюнкции** (или дизъюнкции) и **проекции** (удаления атрибута).

Реляционные аналоги логических операций определяются в терминах отношений на основе обычных теоретико-множественных операций и позволяют выражать напрямую операции пересечения, декартова произведения, естественного соединения, объединения отношений и т. д.

Путем комбинирования базовых операций выражаются операции переименования атрибутов, соединения общего вида, взятия разности отношений. Алгебра А позволяет лучше осознать логические основы реляционной модели, хотя является в меньшей степени ориентированной на практическое применение, чем алгебра Кодда. Даже сами авторы Алгебры А, Дейт и Дарвен, в своем учебном языке Tutorial D используют не Алгебру А напрямую, а некоторое ее надмножество, в большей степени напоминающее алгебру Кодда.

В Алгебре А реляционная операция естественного соединения является одной из базовых операций (действительно естественной), в отличие от алгебры Кодда, где эта операция имела второстепенное значение.

1.4 Свойства операций

Часть операций реляционной алгебры **может** быть выражена через другие операции. Это операции INTERSECT, JOIN, DIVIDE BY.

Примитивными являются операции WHERE, PROJECT, TIMES, UNION, MINUS – они не выражаются через другие операции.

Операции UNION, INTERSECT, TIMES и JOIN в общем случае **ассоциативны** и **коммутативны**.

Можно легко убедиться в том, что операция UNION является ассоциативной. Это означает, что если a, b и c — произвольные отношения одного и того же типа, то следующие два выражения логически эквиваленты:

Пример для операции UNION

Выражение 1

$(a \cup b) \cup c$
 $a \cup b$

Выражение 2

$a \cup (b \cup c)$
 $b \cup a$

Что касается операции TIMES, как показано выше, версия декартова произведения, которая определена в *теории множеств*, не является ни ассоциативной, ни коммутативной, а *реляционная версия* этой операции *обладает обоими этими свойствами*.

Реляционная алгебра введена для обработки отношений и записи реляционных выражений над ними. Основные применения:

1. Определение области для выборки, изменения или удаления данных.
2. Определение правил поддержки целостности
3. Определение производных переменных-отношений

1.5 Реляционное исчисление

Использует понятие **правильно построенной формулы** (well-formed formula), которая служит для выражения условий, накладываемых на переменные отношения. Выделяют реляционное исчисление доменов и кортежей.

1.5.1 Исчисление кортежей

Областями определения переменных исчисления являются **тела кортежей**.

Допустимым значением каждой переменной является кортеж тела некоторого отношения. Правильно построенная формула (wff) включает условия, накладываемые на кортежи, и может содержать:

- Оператор сравнения
- И/ИЛИ/НЕ
- Кванторы (существования и всеобщности)

Набор атрибутов результирующего отношения называется **результатирующим** (целевым) **списком** (target list). Выражением реляционного исчисления является конструкция вида:

target list WHERE WFF

Результатом выполнения операции будет отношение, содержащее все возможные значения кортежей, соответствующих target-list, для которых WFF принимает значение true.

Пример

Примеры с условиями

Groups where (Name = 'M3437' $\vee$ Name = 'M3438')
Students.Name where (length(S.FirstName) =
length(S.LastName) + 3)
S where (FirstName = 'Иван' $\wedge$ LastName $\leq$ 'Иванов')

Примеры с кванторами

G where $\exists S$ (S.FirstName = 'Иван' $\wedge$ S.GId = G.GId)
G where $\forall S$ (S.FirstName = 'Иван' $\vee$ S.GId $\leq$ G.GId)

Использование кванторов $\forall$ и $\exists$ приводит к тому, что значение *следующего за ними* предиката перестает зависеть от конкретных значений соответствующих переменных или от любых дополнительных ограничений, которые могут присутствовать *вне* данного предиката. В этом случае говорят, что переменные *связываются* кванторами.

Понятие *связанной переменной* в определенном смысле аналогично понятию локальной переменной в подпрограмме. Окружающая среда не может повлиять на значение такой переменной, и, следовательно, выполнение подпрограммы не зависит от того, какое значение имела данная переменная в окружении (собственно вне подпрограммы ее и не существует).

Переменные, которые не связаны каким либо квантором, называются *свободными*. От значений свободных переменных в окружении зависит значение соответствующего предиката.

Правильная формула реляционного исчисления кортежей предполагает, что предикат $p(t)$ должен содержать единственную свободную переменную t . Все остальные переменные должны быть связаны.

1.5.2 Исчисление доменов

Областью определения переменных являются *домены*, на которых определены атрибуты отношений. Допустимым значением некоторой переменной является значение некоторого домена.

Основное отличие от исчисления кортежей – это дополнительное множество предикатов, позволяющее выражать условия принадлежности или членства.

Пусть R – n -арное отношение с атрибутами $(A_1, A_2, \dots, A_n)$, то условие принадлежности записывается следующим образом:

$R(a_{i1}:V_{i1}, a_{i2}:V_{i2}, \dots, a_{in}:V_{in})$

где

R – имя отношения,

A – атрибут,

a_{ij} – значения атрибутов,

V – имя переменной домена или const.

Условие принадлежности принимает значение Истина, если в отношении R существует кортеж, содержащий указанные значения указанных атрибутов.

Основным отличием реляционного исчисления кортежей от языка SQL является наличие квантора всеобщности.

2 ЯЗЫК SQL – МАНИПУЛИРОВАНИЕ ДАННЫМИ

Год	Название	Иное название	Изменения
1986	SQL-86	SQL-87	Первый вариант стандарта, принятый институтом ANSI и одобренный ISO в 1987 году.
1989	SQL-89	FIPS 127-1	Немного доработанный вариант предыдущего стандарта.
1992	SQL-92	SQL2, FIPS 127-2	Значительные изменения (ISO 9075); уровень <i>Entry Level</i> стандарта SQL-92 был принят как стандарт FIPS 127-2.
1999	SQL:1999	SQL3	Добавлена поддержка регулярных выражений, рекурсивных запросов, поддержка триггеров, базовые процедурные расширения, нескаллярные типы данных и некоторые объектно-ориентированные возможности.
2003	SQL:2003		Введены расширения для работы с XML-данными, оконные функции (применяемые для работы с OLAP-базами данных), генераторы последовательностей и основанные на них типы данных.
2006	SQL:2006		Функциональность работы с XML-данными значительно расширена. Появилась возможность совместно использовать в запросах SQL и XQuery.
2008	SQL:2008		Улучшены возможности оконных функций, устранены некоторые неоднозначности стандарта SQL:2003
2011	SQL:2011		Реализована поддержка хронологических баз данных (PERIOD FOR), поддержка конструкции FETCH.
2016	SQL:2016		Защита на уровне строк, полиморфные табличные функции, JSON.

2.1 Структура запроса

Каждый запрос к СУБД, сформулированный на языке SQL (Structured Query Language – структурированный язык запросов) представляет собой определенную формулу реляционного исчисления, записанную в SQL – форме. Таким образом, каждый SQL – запрос имеет однозначное отображение в реляционное исчисление и его выполнение может быть представлено операторами реляционной алгебры. Именно переводя запрос в алгебраические операции его выполняет СУБД.

Формальный синтаксис запроса выглядит следующим образом:

```
SELECT список выбора  
[INTO новая таблица]  
FROM исходная таблица  
[WHERE условие поиска]  
[GROUP BY выражение группировки]  
[HAVING условие поиска при группировке]  
[ORDER BY условие сортировки [ASC | DESC]];
```

В квадратных скобках «[элемент запроса]» приведены не обязательные элементы, которые могут отсутствовать в запросе. Обязательными являются только части SELECT и FROM.

В самых простых вариантах запрос может быть представлен, как показано в примере 1.8.

Пример 1.8.(a)

```
select *  
from TDoljnost;
```

Пример 1.8.(b)

```
select Doljnost, Oklad  
from TDoljnost;
```

В примере 1.8.(a) выбираются все столбцы из таблицы TDoljnost, в примере 1.8.(b) – только столбцы Doljnost и Oklad.

Операции языка SQL являются замкнутыми, как и операции реляционной алгебры. Это означает, что в результате выполнения операции над отношением будет получено, в общем случае, другое отношение.

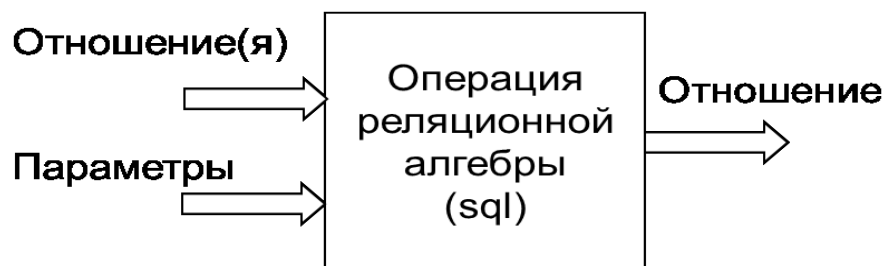


Рисунок. Замкнутость операций реляционной алгебры

2.2 Запросы с условием

Часть WHERE накладывает условие на результат запроса и, как правило, интерпретируется как операция сокращения (выборки). В составлении условия могут быть задействованы обычные условные операторы и несколько специальных:

= - равно;

<> - (или != в зависимости от диалекта¹ SQL) не равно;

< - меньше;

> - больше;

>= - больше чем или равно;

<= - меньше чем или равно;

!> - не больше;

!< - не меньше;

IN и BETWEEN – определяют, принадлежит ли элемент множеству, которое может быть задано реляционным выражением (другим запросом);

Like – определяет схожесть строк, с учетом N-грамм²;

AND – и, логическая конъюнкция;

NOT – не, логическое отрицание;

OR - или, логическая дизъюнкция;

[NOT] EXISTS – существование (более подробно см. [4]);

IS [NOT] NULL – имеет значение null.

Пример запроса с условием приведен в примере 1.9.

Пример 1.9.

```
select Doljnost, Oklad
from TDoljnost
where (Oklad*0.87<40000)
and
(Doljnost like N'%нженер%')
```

Конструкция GROUP BY позволяет проводить группировку значений результата выполнения операции. При этом необходимо следить за корректным списком столбцов результата. Пример использования группировки без наложения условия на группы и с условием приведен в примере 1.10.

Пример 1.10.(a)

```
select Doljnost as 'Должность'
from TEmployers
group by Doljnost;
```

¹ Каждый производитель СУБД разрабатывает собственные расширения для языка, помимо поддерживаемого стандарта. Таким образом, на основе базового языка SQL появляются различные «диалекты» от различных производителей реляционных СУБД.

² N-грамма это последовательность из n элементов, включающая служебные символы (замена одного любого символа, все оставшиеся символы и т.д.

Результат выполнения:

Должность

Ведущий инженер
Главный инженер
Инженер
Специалист

Пример 1.10.(b)

```
select Doljnost as 'Должность'  
from TEmployers  
group by Doljnost  
having Doljnost like N'%нженер%';
```

Результат выполнения:

Должность

Ведущий инженер
Главный инженер
Инженер

Обратите внимание, что во втором примере благодаря условию `Doljnost like N'%нженер%'`, показывающему, что, так как выбираются из базы данных все инженерные должности, название должности должно содержать «нженер» с любыми символами до и после, должность «специалист» была исключена из результата.

Также в этом примере показана операция переименования столбцов результата `as 'Новое_название'`. Она используется для удобства чтения результата, переименования одинаковых столбцов, если такие могут оказаться в результате, и задания имен столбцам с агрегирующими функциями.

2.3 Запросы с агрегированием

Основными агрегирующими функциями являются:

MIN –наименьшее значение из заданного множества;

MAX - определяет наибольшее из заданного множества;

SUM – определяет сумма значений из заданного множества;

AVG – определяет среднее значение по заданному множеству;

COUNT (COUNT_BIG) – количество значений в заданном множестве. Это единственная функция, учитывающая неопределенные значения null в своей работе.

В общем случае фактическому параметру агрегирующей функции может предшествовать необязательное ключевое слово DISTINCT (например, SUM (DISTINCT имя_поля)), которое указывает, что перед применением этой функции дублирующиеся строки должны быть удалены. Для ряда функций (например, MAX и MIN) ключевое слово DISTINCT логично является излишним и не вызывает никакого действия.

Специальная агрегирующая функция COUNT(*) **не допускает использования ключевого слова DISTINCT** и предназначена для подсчета всех строк в таблице **без предварительного удаления дублирующихся строк**.

Если после удаления неопределенных значений должно остаться пустое множество, то агрегирующая функция COUNT возвращает нуль, а все остальные операторы возвращают неопределенное значение.

Использование агрегирующих функций проиллюстрировано примером 1.11. для функции COUNT и 1.12. для MIN.

Пример 1.11.

```
select Doljnost as 'Должность',  
       COUNT(*) as 'Число сотрудников'  
from TEmployers  
group by Doljnost;
```

Пример 1.12.

```
select Doljnost as 'Должность',  
       MIN(SocialNumber) as  
       'Наименьший соц. номер'  
from TEmployers  
group by Doljnost;
```

Обратите внимание, что агрегирующие функции несут в себе элемент обобщения и применяются (как правило) вместе с группировкой.

Ряд диалектов языка вводят свои агрегирующие функции. Этот список также пополняется с развитием стандарта SQL.

2.4 Запросы с соединением. Виды соединений

Важным элементом SQL-запросов является соединение (JOIN). В отличие от реляционного подхода, язык SQL в соответствии со стандартом предлагает несколько различных видов соединений: собственно, соединение, иногда также называемое внутренним или естественным (JOIN, NATURAL JOIN, INNER JOIN в разных диалектах), и различные виды внешних соединений: LEFT JOIN (левое внешнее соединение), RIGHT JOIN (правое внешнее соединение), FULL JOIN (или, чаще, CROSS JOIN) – полное внешнее соединение (декартово произведение). Использование внутреннего соединения рассмотрено в примере 1.13., вместе с примером данных и **результата** соединения.

Пример 1.13.

```
select TEmployers.Employer, TEmployers.Doljnost,  
       TDoljnost.Oklad  
from TDoljnost inner join TEmployers on  
       TEmployers.Doljnost=TDoljnost.Doljnost
```

Рассмотрим работу этого примера на данных. Исходные отношения TDoljnost и TEmployers, а также результат их соединения приведены на рисунке 1.10.

Обратите внимание на выделенные записи. Должности «Администратор» и «Менеджер» отсутствуют в таблице сотрудников и из результирующего отношения записи о них исключены. Должность у сотрудника Кречетова не заполнена (имеет значение Null), и из результата этот сотрудник также исключен.

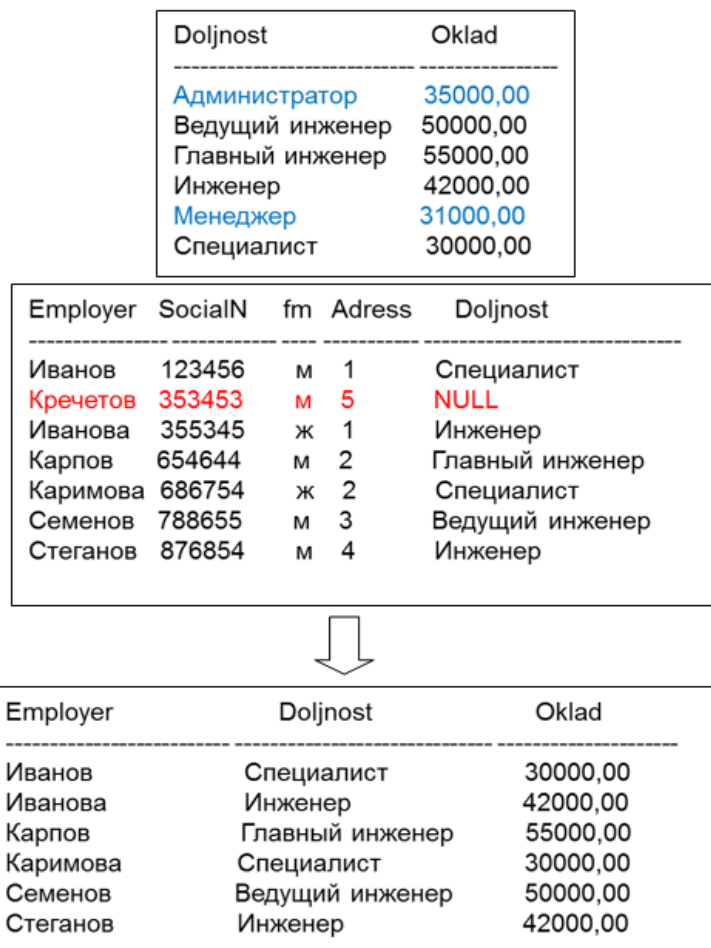


Рисунок 1.10. Использование внутреннего соединения

В остальном результат операции представляет собой конкатенацию кортежей TDoljnost и TEmployers с одинаковыми значениями Должности за счет условия в запросе «on TEmployers.Doljnost=TDoljnost.Doljnost», показывающего, по каким полям проводится соединение. Список результирующих столбцов, как и раньше, определяется частью оператора SELECT: TEmployers.Employer, TEmployers.Doljnost, TDoljnost.Oklad. Обратите также внимание, что в этом случае отношение, из которого взят атрибут результата, указывается явно.

Надо отметить, что запрос, приведенный в примере 1.13 аналогичен запросу в примере 1.14, однако запись формата 1.13 является более предпочтительной, так как **в явном виде** указывает соединение отношений и по каким атрибутам оно происходит.

Пример 1.14.

```
select TEmployers.Employer, TEmployers.Doljnost,
      TDoljnost.Oklad
from TEmployers, TDoljnost
where TEmployers.Doljnost=TDoljnost.Doljnost
```

В примере 1.14 равенство значений столбцов, по которым должно происходить соединение, указано в части WHERE.

При отсутствии совпадающих столбцов соединение JOIN выполняется как декартово произведение (пример 1.15).

Пример 1.15.

```
select TEmployers.Employer, TEmployers.Doljnost,
```

```

TDoljnost.Oklad
from TDoljnost full join TEmployers
on TEmployers.Doljnost=TDoljnost.Doljnost

```

Операнды и результат приведен на рисунке 1.11.

Обратите внимание, что в этом случае строки из первого и второго отношений не вошедшие в результат обычного (внутреннего) соединения также представлены в результате полного соединения.

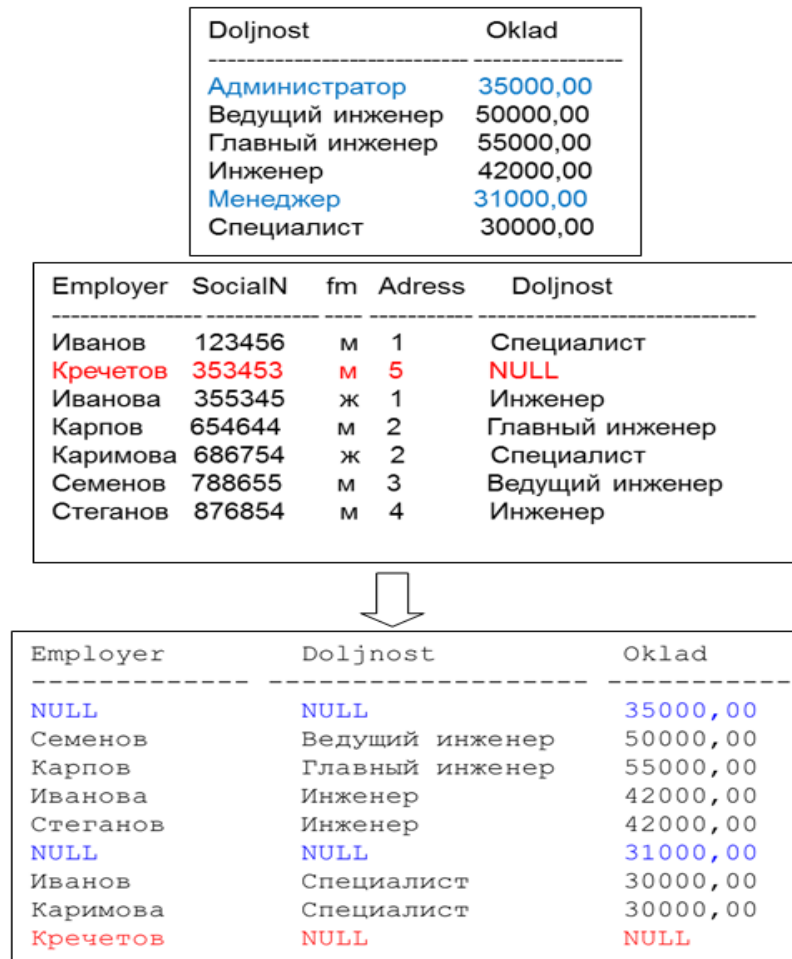


Рисунок 1.11. Полное внешнее соединение

Левое и правое внешние соединения работают несколько по-другому. Рассмотрим пример правого соединения. Код запроса приведен в примере 1.16.

Пример 1.16.

```

select TEmployers.Employer, TEmployers.Doljnost,
TDoljnost.Oklad
from TDoljnost right join TEmployers on
TEmployers.Doljnost=TDoljnost.Doljnost

```

Результатом такого запроса будет отношение, представленное на рисунке 1.12. Отметим, что в качестве операндов использовались отношения из предыдущих примеров внутреннего и полного внешнего соединения.

Employer	Doljnost	Oklad
Иванов	Специалист	30000,00
Кречетов	NULL	NULL
Иванова	Инженер	42000,00
Карпов	Главный инженер	55000,00
Каримова	Специалист	30000,00
Семенов	Ведущий инженер	50000,00
Стеганов	Инженер	42000,00

Рисунок 1.12. Правое внешнее соединение

В этом случае в результат вошли части всех кортежи второго отношения, и соответствующие им части кортежей из первого. Для левого соединения ситуация выглядит обратной (пример 1.17).

Пример 1.17.

```
select TEmployers.Employer, TEmployers.Doljnost,
       TDoljnost.Oklad
from TDoljnost left join TEmployers on  TEmployers.Doljnost=TDoljnost.Doljnost
```

Результат для тех же операндов, что и в предыдущем примере, представлен на рисунке 1.13. В нем сохранены все кортежи (их части, в соответствии с условиями части SELECT) первого отношения и добавлены соответствующие части кортежей второго при соединении по полю «Должность».

Employer	Doljnost	Oklad
NULL	NULL	35000,00
Семенов	Ведущий инженер	50000,00
Карпов	Главный инженер	55000,00
Иванова	Инженер	42000,00
Стеганов	Инженер	42000,00
NULL	NULL	31000,00
Иванов	Специалист	30000,00
Каримова	Специалист	30000,00

Рисунок 1.13. Левое внешнее соединение

Также в силу замкнутости реляционной алгебры и языка SQL, о которых было сказано выше, можно использовать вложенные подзапросы и выполнять операции объединения результатов запросов. Вложенные подзапросы иллюстрируются примером 1.18.

Пример 1.18.

```
select EventID, EDescription
from TEvent
where EventID in
      (select EventID from EventStruct where ELevel=0)
```

В данном примере выбираются идентификатор и описание события, если идентификатор события принадлежит множеству, определяемому подзапросом. В общем случае, без учета оптимизации, для каждого значения EventID будет отдельно вычислено значение подзапроса. Очевидно, в данном, как и во многих других случаях, запрос может быть переписан без использования вложенности.

Объединение результатов двух запросов при помощи оператора UNION представлено в примере 1.19.

Пример 1.19

```
select Employer, Doljnost, fm
  from TEmployers
 where fm=N'ж' and Doljnost like N'%нженер%'
union
select Employer, Doljnost, fm
  from TEmployers
 where fm=N'м' and (Doljnost=N'Менеджер'
                    or Doljnost=N'Администратор'
                    or Doljnost=N'Специалист');
```

Обратите внимание, что заголовки отношений, участвующих в операции объединения должны совпадать – аналогично соответствующей реляционной операции.